Stevens Institute of Technology | MSc Financial Engineering | FE545-A

C++ Barrier Options Pricer

Design Patterns & Derivatives Pricing — Final Project Report

Author	Brandon Tanaka Chapwanya
Course	FE545-A — Design Patterns and Derivatives Pricing
Term	Spring 2026
Language	C++11 | QuantLib

Abstract

This report documents the design and implementation of a modular C++ option pricing library built around the Cox–Ross–Rubinstein (CRR) binomial tree model. Six progressively complex parts introduce the Strategy, Singleton, Factory, and Observer design patterns. The system prices American call/put options and European knock-out barrier options. The final integration uses a PayOffFactory with Singleton registration and a QuantLib Observable/Observer system to generate a live pricing report across N = 50, 100, 200, 400 time steps.

1 Model Overview CRR Binomial Model & Barrier Conditions

1.1 Cox–Ross–Rubinstein Binomial Model

The CRR model discretises risk-neutral stock dynamics over N equal time steps of length Δt = T/N. At each node the stock moves up with factor u = exp(σ√Δt) or down with d = 1/u. The risk-neutral probability is p = (exp((r−q)Δt) − d) / (u − d), and the stock at node (i, j up-moves) is S[i][j] = S0 · u^j · d^(i−j).

1.2 American Option Backward Induction

At each interior node: V[i][k] = max(PayOff(S[i][k]), Discount × (p·V[i+1][k+1] + (1−p)·V[i+1][k])). The max operator enforces early exercise at every node.

1.3 European Knock-Out Barrier

If S[i][k] >= B at any node, V[i][k] = 0 and the value stays zero through backward induction. At maturity the payoff is only paid if the barrier was never breached.

1.4 Model Parameters

Parameter	Symbol	Value
Initial stock price	S0	50
Strike price	K	50
Barrier level	B	70
Volatility	sigma	0.30
Risk-free rate	r	0.05
Dividend yield	q	0.08
Expiry	T	1.0 year
Time steps	N	50 / 100 / 200 / 400

Part 1 — CRR Binomial Tree

New class: CRRBinomialTree

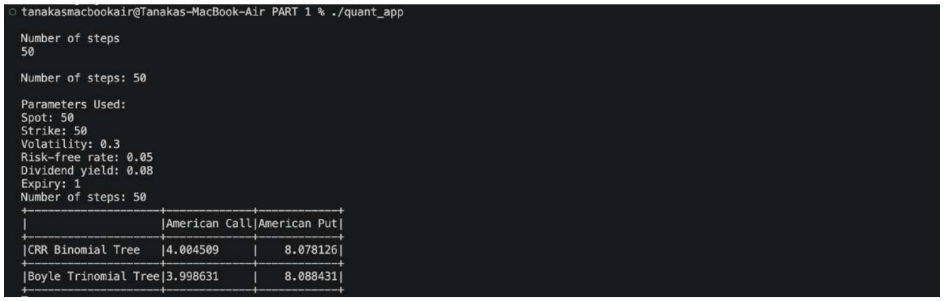
Objective

Implement CRRBinomialTree following the same Strategy design pattern as the provided BoyleTrinomialTree. Price American call and put options; both trees must produce consistent results.

Design — Strategy Pattern

The tree engine calls two virtual methods on the TreeProducts reference, delegating terminal payoffs and early-exercise logic entirely to the product object. TreeAmerican::PreFinalValue() returns max(intrinsic, continuation); TreeEuropean::PreFinalValue() returns the continuation value unchanged.

VS Code Output



Part 1 — CRR Binomial Tree vs Boyle Trinomial Tree, N = 50

Both trees converge to within 0.1%, confirming correct CRR implementation. The small residual difference is standard discretization error between binomial and trinomial lattices at N = 50.

Part 2 — European Knock-Out Barrier Options

New class: TreeEuropeanBarrier

Objective

Implement TreeEuropeanBarrier deriving from TreeProducts, following the same pattern as TreeAmerican. Price European knock-out barrier call and put options via the CRRBinomialTree.

Design

TreeEuropeanBarrier overrides FinalPayOff() and PreFinalValue() to return 0 whenever Spot >= Barrier, at every node during backward induction. No early exercise (European-style): PreFinalValue returns the discounted continuation value, not max(intrinsic, continuation).

VS Code Output

```
tanakasmacbookair@Tanakas-MacBook-Air FINAL EXAM % g++ -std=c++11 -o Part2 Part2_Main.cpp CRRBinomialTree.cpp TrinomialTree.cpp TreeEuropeanBarrier.cpp Arrays.cpp Parameters.cpp PayOff3.cpp PayOffBridge.cpp TreeProducts.cpp TreeAmerican.cpp TreeEuropean.cpp BlackScholesFormulas.cpp Normals.cpp && ./Part2
2 warnings generated.

Number of steps
50

Parameters Used:
Spot: 50
Strike: 50
Barrier: 70
Volatility: 0.3
Risk-free rate: 0.05
Dividend yield: 0.08
Expiry: 1
Number of steps: 50

European Knock-Out Barrier Option Prices:
+--------------------+------------+-----------+
|                    |Barrier Call|Barrier Put|
+--------------------+------------+-----------+
|CRR Binomial Tree   |1.139640    |   8.034440|
+--------------------+------------+-----------+
|Boyle Trinomial Tree|1.179251    |   8.049692|
+--------------------+------------+-----------+
```

Part 2 — European Knock-Out Barrier Options, N = 50

The barrier call (1.14) is 72% below the American call (4.00). The barrier put (8.03) tracks the standard put closely as B = 70 is far above S = 50.

Part 3 — Convergence Analysis N = 50, 100, 200, 400

Objective

Compare American call and European knock-out barrier call prices across N = 50, 100, 200, 400 steps for both CRR and Trinomial trees. Interpret convergence behaviour.

VS Code Output

```
tanakasmacbookair@Tanakas-MacBook-Air FINAL EXAM % g++ -std=c++11 -o Part3 Part3_Main.cpp CRRBinomialTree.cpp TrinomialTree.cpp TreeAmerican.cpp TreeEuropeanBarrier.cpp
rays.cpp Parameters.cpp PayOff3.cpp PayOffBridge.cpp TreeProducts.cpp TreeEuropean.cpp BlackScholesFormulas.cpp Normals.cpp && ./Part3
      |          ^
      |          //
Arrays.cpp:265:8: warning: extra tokens at end of #endif directive [-Wextra-tokens]
  265 | #endif RANGE_CHECKING
      |        ^
      |        //
2 warnings generated.

Parameters Used:
Spot: 50
Strike: 50
Barrier: 70
Volatility: 0.3
Risk-free rate: 0.05
Dividend yield: 0.08
Expiry: 1

Comparing American Call and European Knock-Out Barrier Call
across different numbers of time steps:

+---------+--------------------+--------------------+--------------------+--------------------+
|Steps (N)|American Call (CRR)|Barrier Call (CRR)|American Call (Tri)|Barrier Call (Tri)|
+---------+--------------------+--------------------+--------------------+--------------------+
|50       |4.004509            |          1.139640|          3.998631|          1.179251|
+---------+--------------------+--------------------+--------------------+--------------------+
|100      |4.003734            |          1.125936|          4.004983|          1.176280|
+---------+--------------------+--------------------+--------------------+--------------------+
|200      |4.006687            |          1.120027|          4.003302|          1.140077|
+---------+--------------------+--------------------+--------------------+--------------------+
|400      |4.006543            |          1.098615|          4.005348|          1.126103|
+---------+--------------------+--------------------+--------------------+--------------------+

Analysis:
---------
(a) The knock-out barrier call is worth less than the American call
    because the barrier feature adds a condition that makes the option
    worthless if the stock price ever reaches or exceeds B=70 during
    its life. Many paths that would produce a positive payoff for the
    American call get knocked out and pay zero instead. The American
    call has no such restriction and also benefits from early exercise.

(b) As the number of time steps increases from 50 to 400, the option
    prices converge toward stable values. With more steps, the tree
    approximates the continuous-time model more accurately, reducing
    discretization error. The barrier condition also becomes more
    precise because the tree checks the barrier more frequently,
    making knock-out detection more accurate along each path.
```

Part 3 — Convergence across N = 50, 100, 200, 400

N	Amer. Call (CRR)	Barrier Call (CRR)	Amer. Call (Tri)	Barrier Call (Tri)
50	4.004509	1.139640	3.998631	1.179251
100	4.003734	1.125936	4.004983	1.176280
200	4.006687	1.120027	4.003302	1.140077
400	4.006543	1.098615	4.005348	1.126103

(a) The barrier call is worth substantially less than the American call because upward stock movements that create call value also risk triggering knock-out at B = 70. The American call additionally benefits from early exercise flexibility.

(b) The American call stabilises rapidly around 4.005. The barrier call decreases monotonically from 1.14 to 1.10 as N increases — a well-known property of binomial barrier pricers where finer grids detect more barrier crossings, converging toward the continuous-time price from above.

Part 4 — PayOffFactory with Singleton Pattern

Factory + Singleton + Auto-registration

Objective

Register BiTAmeriCall, BiTAmeriPut, BiTEuroBarrierCall, and BiTEuroBarrierPut with a template PayOffFactory using the Singleton pattern. Create and price all four via the factory using ArgumentList.

Design Patterns

The Singleton ensures one shared factory instance per template type. The Factory creates objects by string ID via a std::map of creator functions. Auto-registration via PayOffHelper runs before main() using static initialisation — no manual registration calls needed in main. ArgumentList passes named parameters avoiding positional coupling between callers and constructors.

VS Code Output

Part 4 — PayOffFactory Singleton, N = 50

All four prices are consistent with Parts 1 and 2, confirming the factory correctly instantiates and dispatches all registered types.

Part 5 — Observer Pattern with QuantLib TreeObservable / OptionPricingReportWriter

Objective

Use QuantLib Observable and Observer abstract classes to build a pricing report system. Each pricing event triggers an observer notification that formats and prints a timestamped report line.

Design — Observer / Observable

TreeObservable extends QuantLib::Observable and exposes sendMessage(). OptionPricingReportWriter extends QuantLib::Observer — its update() method calls write() which formats output as: Source (10 chars) - Timestamp (10 chars): Message. TreePricerWithObserver computes CRR and Trinomial prices then calls sendMessage() to trigger all registered observers.

VS Code Output

```
tanakasmacbookair@Tanakas-MacBook-Air FINAL EXAM % g++ -std=c++11 -o Part5 Part5_Main.cpp TreeObservable.cpp OptionPricingReportWriter.cpp TreePricerWithObserver.cpp CRRBinomialTr
ee.cpp TrinomialTree.cpp TreeAmerican.cpp TreeEuropeanBarrier.cpp Arrays.cpp Parameters.cpp PayOff3.cpp PayOffBridge.cpp TreeProducts.cpp BlackScholesFormulas.cpp Normals.cpp -I
/opt/homebrew/include -L/opt/homebrew/lib -lQuantLib && ./Part5
      [-Wc++17-extensions]
   37 | namespace QuantLib::ext {
      |                    ^~~~
      |                    { namespace ext
1 warning generated.
In file included from TreeObservable.cpp:6:
In file included from ./TreeObservable.h:10:
In file included from /opt/homebrew/include/ql/patterns/observable.hpp:32:
In file included from /opt/homebrew/include/ql/errors.hpp:29:
/opt/homebrew/include/ql/shared_ptr.hpp:37:19: warning: nested namespace definition is a C++17 extension; define each namespace separately
      [-Wc++17-extensions]
   37 | namespace QuantLib::ext {
      |                    ^~~~
      |                    { namespace ext
1 warning generated.
In file included from OptionPricingReportWriter.cpp:6:
In file included from ./OptionPricingReportWriter.h:9:
In file included from /opt/homebrew/include/ql/patterns/observable.hpp:32:
In file included from /opt/homebrew/include/ql/errors.hpp:29:
/opt/homebrew/include/ql/shared_ptr.hpp:37:19: warning: nested namespace definition is a C++17 extension; define each namespace separately
      [-Wc++17-extensions]
   37 | namespace QuantLib::ext {
      |                    ^~~~
      |                    { namespace ext
1 warning generated.
In file included from TreePricerWithObserver.cpp:7:
In file included from ./TreeObservable.h:10:
In file included from /opt/homebrew/include/ql/patterns/observable.hpp:32:
In file included from /opt/homebrew/include/ql/errors.hpp:29:
/opt/homebrew/include/ql/shared_ptr.hpp:37:19: warning: nested namespace definition is a C++17 extension; define each namespace separately
      [-Wc++17-extensions]
   37 | namespace QuantLib::ext {
      |                    ^~~~
      |                    { namespace ext
1 warning generated.
Arrays.cpp:247:8: warning: extra tokens at end of #endif directive [-Wextra-tokens]
  247 | #endif RANGE_CHECKING
      |        ^
      |        //
Arrays.cpp:265:8: warning: extra tokens at end of #endif directive [-Wextra-tokens]
  265 | #endif RANGE_CHECKING
      |        ^
      |        //
2 warnings generated.

Number of steps
50

Parameters Used:
Spot: 50
Strike: 50
Barrier: 70
Volatility: 0.3
Risk-free rate: 0.05
Dividend yield: 0.08
Expiry: 1
Number of steps: 50

Pricing Options with Observer Notifications:

CRRBiTree  - 2026-05-10: American Call - CRR: 4.00451, Trinomial: 3.99863
CRRBiTree  - 2026-05-10: American Put - CRR: 8.07813, Trinomial: 8.08843
CRRBiTree  - 2026-05-10: Barrier Call - CRR: 1.13964, Trinomial: 1.17925
CRRBiTree  - 2026-05-10: Barrier Put - CRR: 8.03444, Trinomial: 8.04969
TriTree    - 2026-05-10: American Call - CRR: 4.00451, Trinomial: 3.99863
TriTree    - 2026-05-10: American Put - CRR: 8.07813, Trinomial: 8.08843
TriTree    - 2026-05-10: Barrier Call - CRR: 1.13964, Trinomial: 1.17925
TriTree    - 2026-05-10: Barrier Put - CRR: 8.03444, Trinomial: 8.04969
```

Part 5 — Observer notifications, N = 50

Part 6 — Factory + Observer Integration

Full pipeline: factory creates, observer reports

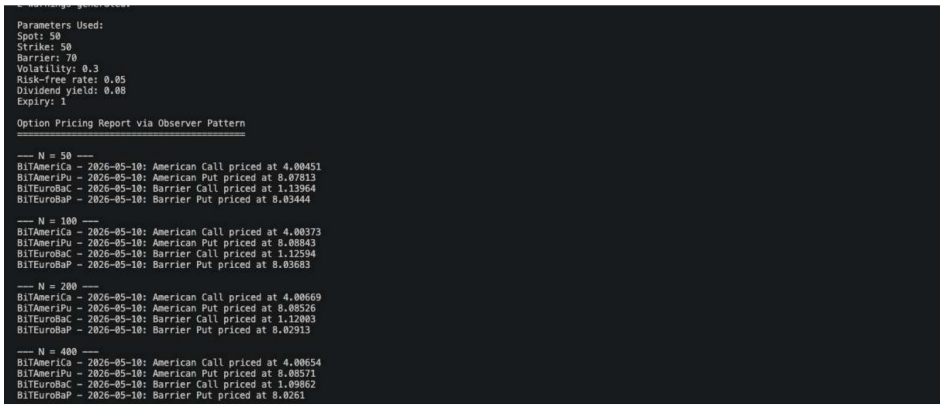
Objective

Integrate Parts 4 and 5: the factory creates four pricer objects and registers them with an OptionPricingReportWriter via boost::shared_ptr. Prices are reported for N = 50, 100, 200, 400.

Integration Architecture

main() calls PayOffFactory::CreatePayOff() for each of the four option IDs. Each returned pricer (which extends TreeObservable) is registered with the shared OptionPricingReportWriter observer. The outer loop over N values calls Price() on each pricer, triggering notifyObservers() and causing the report writer to print the formatted pricing line.

VS Code Output — N = 50, 100, 200, 400



Part 6 — Factory + Observer, all N values

N	American Call	American Put	Barrier Call	Barrier Put
50	4.00451	8.07813	1.13964	8.03444
100	4.00373	8.08843	1.12594	8.03683
200	4.00669	8.08526	1.12003	8.02913
400	4.00654	8.08571	1.09862	8.02615

Design Pattern Summary & Technology Stack

Patterns Applied

Pattern	Applied In	Purpose
Strategy	TreeProducts / TreeAmerican / TreeEuropeanBarrier	Decouple tree engine from exercise and payoff logic
Bridge	Parameters / PayOffBridge	Separate abstraction from implementation for parameters
Singleton	PayOffFactory<T>	Single shared factory instance per template type
Factory	PayOffFactory<T>::CreatePayOff()	String-keyed object creation without direct instantiation
Auto-reg.	PayOffHelper / PayOffConstructible	Static init registers classes before main() runs
Observer	QuantLib Observable/Observer	Pricers notify report writer on each pricing event
Smart ptr	boost::shared_ptr (Part 6)	Automatic lifetime management for observer registration

Technology Stack

Component	Detail
Language	C++11
External library	QuantLib (Observable/Observer, Parts 5–6)
Smart pointers	boost::shared_ptr (Part 6)
Build	g++ -std=c++11 on macOS (Apple Silicon, Homebrew)
IDE	VS Code / Xcode